Applied Type Theory

Лекция 1: введение в λ-исчисление

Воронов Михаил Сергеевич

ВМК МГУ · Осень 2026

План лекции

1. Оргчасть и устройство курса
2. Мотивация
3. Введение в λ-исчисление
4. Формальный синтаксис и нотация
5. Редукции и отношения эквивалентности
6. Комбинаторы и кодирование данных

План лекции

1 **Оргчасть и устройство курса**

2 Мотивация

3 Введение в λ-исчисление

4 Формальный синтаксис и нотация

5 Редукции и отношения эквивалентности

6 Комбинаторы и кодирование данных

Теоретический план курса

- Нетипизированное λ-исчисление →
  просто типизированное λ-исчисление → алгебра типов.
- Классические логики и исчисления: NJ / LK.
- System F ($\lambda 2$), $\lambda\omega$, зависимые типы: MLTT / HoTT.
- Субструктурные типы: линейные и аффинные, их связь с Rust и Haskell.

Практический план курса

- Теоретические задачи на λ-исчисление и системы типов.
- Практические задачи на Coq и OCaml, дополнительные примеры на Agda и TLA+.
- Объяснение условий и следствий теорем простым языком.
- Задачи о корректности AI-компонентов: формы тензоров, типизированные конвейеры, проверяемые доказательства.
- Проектирование API на базе теории типов.
- Обязательные задачи и задачи со звёздочкой.
  Последние принимаются до конца семестра.

Критерии оценивания

5 домашних заданий — 293 балла по обязательным задачам.
За экзамен можно получить до 150 баллов.

Сумма баллов	Оценка
0–101	2
102–189	3
190–262	4
263 и выше	5

Литература

- Pierce — *Types and Programming Languages*.
- Nederpelt, Geuvers — *Type Theory and Formal Proof*.
- *Software Foundations*, тома 1–2.
  softwarefoundations.cis.upenn.edu
- Bertot, Castéran — *Interactive Theorem Proving and Program Development*.
- Mimram — *Program = Proof*.
- Курс Д. Н. Москвина «Функциональное программирование» (YouTube).

План лекции

1 Оргчасть и устройство курса

2 Мотивация

3 Введение в λ-исчисление

4 Формальный синтаксис и нотация

5 Редукции и отношения эквивалентности

6 Комбинаторы и кодирование данных

Зачем это разработчику AI

Нейросимвольный подход

Нейросеть предлагает программу или доказательство, а символический компонент проверяет заданные свойства.

- **Типы как контракты:** задают данные, состояния API и допустимые переходы.
- **Раннее обнаружение ошибок:** проверка размеров тензоров и аргументов функций.
- **Формальная верификация:** проверка свойств сгенерированного кода по спецификации.
- **Обратная связь:** ошибка проверки или незакрытая цель направляет поиск модели.
- **Граница гарантий:** типы и доказательства относятся к записанной модели задачи.

Примеры использования типовых техник

Задача	Типовая техника
Отсутствующий результат	`Option`/`Maybe` и обработка отсутствия значения
Несовместимые размеры тензоров	Индексированные типы `Matrix[m, n]`
Нарушение протокола API	Typestate и сессионные типы
Повторное освобождение буфера	Линейные/аффинные типы и владение
Пропущенный вариант действия	ADT и исчерпывающий pattern matching
Неверная структура генерируемого кода	Типизированные AST/DSL
Смешение доменов данных	Отдельные типы `TokenId`, `Position` и `ModelId`
Нарушение свойств алгоритма	Зависимые типы и доказательства спецификаций

API как контракт: typestate

Основная идея

Состояние объекта выражается в типе. Сигнатуры задают допустимые переходы.

Пример: состояния модели (псевдокод)

```
type Untrained
type Trained
type Model[State]

train   : Model[Untrained] × Dataset → Model[Trained]
predict : Model[Trained] × Input → Prediction
```

- `predict` для `Model[Untrained]` не типизируется: нужен `Model[Trained]`.
- `Trained` фиксирует этап API, но не доказывает качество предсказаний.

Ресурсы и память: линейные типы

Контракты ресурсов

Линейный ресурс используется ровно один раз, аффинный — не более одного. Пример: право владения GPU-буфером.

Схема API: $\multimap$ означает потребление ресурса

$$\begin{aligned} \text{alloc} &: \text{Shape} \to \text{Buffer} \\ \text{compute} &: \text{Buffer} \multimap \text{Buffer} \\ \text{release} &: \text{Buffer} \multimap \text{Unit} \end{aligned}$$

- После `release` право владения потреблено. Повторное освобождение не типизируется.
- Правила заимствования ограничивают совместный доступ с записью.

Протоколы как типы

Сессионные типы

Agent : !Req. ?Result. end

Tool : ?Req. !Result. end

`!` — отправка, `?` — получение,
`end` — завершение сеанса.

Тип задаёт порядок обмена: запрос к инструменту, результат, завершение.

Двойственность типов согласует отправку и получение сообщений.

Инвариант состояния

Handled ⊆ Sent

`Sent` — id отправленных запросов.
`Handled` — id обработанных ответов.

Ответ относится к ранее отправленному запросу. Это свойство проверяется в модели состояний.

Для гарантии ответа нужны дополнительные предположения о сети и инструменте.

Формальная верификация: что будем доказывать

- Свойства алгоритмов в Coq: результат сортировки упорядочен и сохраняет все элементы входа с их повторениями.

$$ys = \mathsf{sort}(xs) \quad \Rightarrow \quad \mathsf{Sorted}(ys) \wedge \mathsf{Permutation}(xs, ys)$$

- Инварианты данных: длины списков, допустимость индекса, размеры тензоров.
- Законы полиморфного кода и их предпосылки: например, $\mathsf{length}(\mathsf{map}\ f\ xs) = \mathsf{length}(xs)$.

Практический эффект

Нейросеть может предложить программу и доказательство. Проверяющее ядро принимает завершённое доказательство записанной спецификации.

Ошибка проверки возвращается модели как обратная связь.

Ограничения и зона ответственности

- **Типы структуры данных** не гарантируют смысл ответа: валидный JSON может содержать неверное утверждение или неподходящее действие.
- **Формальная спецификация** должна отражать задачу. Доказательство неверно выбранного свойства не подтверждает нужное поведение.
- **Размеры тензоров и состояния API** не определяют качество модели, качество данных и устойчивость к изменению распределения.
- **Доказательства** действуют в принятой модели и при её предположениях. Внешние сервисы, оборудование и небезопасный код требуют отдельного анализа.
- **Нейросетевой поиск** может не найти решение. Наличие проверяющего компонента само по себе не гарантирует завершение поиска.

От программы к доказательству

Пример: тождественная функция

$$\lambda x.x \ : \ A \to A$$

- **Вычисление:** функция возвращает свой аргумент.
- **Тип:** $A \to A$ задаёт аргумент типа A и результат того же типа.
- **Логика:** $A \to A$ выражает утверждение «из A следует A».

Роль нейросети

Нейросеть ищет терм заданного типа.
Формальная система проверяет построенный терм.

План лекции

1 Оргчасть и устройство курса

2 Мотивация

3 Введение в λ-исчисление

4 Формальный синтаксис и нотация

5 Редукции и отношения эквивалентности

6 Комбинаторы и кодирование данных

λ-исчисление

- λ-исчисление — формальная система, разработанная Алонзо Чёрчем в 1930-х для формализации и анализа понятия вычислимости.
- Имеет нетипизированную (untyped) и множество типизированных версий.
- Позволяет описывать семантику вычислительных процессов.
- Является теоретической основой для многих помощников доказательства.

Поведение функций

Рассмотрим функцию $f(x) = x^2 + 1$.

- Эта функция имеет один «вход» и один «выход»: $f : \mathbb{R} \to \mathbb{R}$.
- Её можно рассматривать как отображение $x \mapsto x^2 + 1$.
- Чтобы подчеркнуть роль параметра x, используют специальный символ λ:

$$\lambda x.x^2 + 1$$

- Здесь x — параметр функции, а не конкретное число.
- Для конкретного значения можно применить функцию: $(\lambda x.x^2 + 1)(3)$.

Введение в λ-исчисление

Для работы с функциями используются два способа построения выражений:

1. Абстракция: из выражения M и переменной x можно составить новое выражение $\lambda x.M$ — абстракцию по x.
2. Применение: из двух выражений M и N можно составить новое выражение $M\,N$.

Введение в λ-исчисление: абстракция

1. Пусть $M = M[x]$ — выражение,
   возможно содержащее свободные вхождения x.
2. Тогда абстракция $\lambda x.M$ обозначает функцию $x \mapsto M[x]$.
3. Абстракция — способ задать неименованную функцию.
4. Если $x \notin \mathrm{FV}(M)$, то $\lambda x.M$ — константная функция со значением M.

Введение в λ-исчисление: применение

1. Применение F к X можно понимать как выполнение алгоритма F на данных X.
2. Явного различия между алгоритмами и данными нет. В частности, возможно самоприменение $F\,F$.
3. Применение абстракции вычисляется β-преобразованием:

$$(\lambda x.M)\,N \to_\beta M[x := N]$$

4. $M[x := N]$ — терм M, в котором свободные вхождения x заменены на N без захвата переменных.

Введение в λ-исчисление: примеры β-редукции

$(\lambda x.x^2 + 1)\, 3 \to_\beta 3^2 + 1$

$(\lambda y.5)\, 1 \to_\beta 5$

$(\lambda x.x)\, (\lambda y.y) \to_\beta \lambda y.y$

$\lambda z.(\lambda x.x)\, (\lambda y.y) \to_\beta \lambda z.\lambda y.y$

План лекции

1 Оргчасть и устройство курса

2 Мотивация

3 Введение в λ-исчисление

4 Формальный синтаксис и нотация

5 Редукции и отношения эквивалентности

6 Комбинаторы и кодирование данных

Формальное построение λ-исчисления

Определение

Множество λ-термов Λ строится индуктивно. V — бесконечное множество переменных.

$$x \in V \;\Rightarrow\; x \in \Lambda$$

$$M, N \in \Lambda \;\Rightarrow\; (M\,N) \in \Lambda$$

$$M \in \Lambda,\; x \in V \;\Rightarrow\; (\lambda x.M) \in \Lambda$$

- Абстракция $\lambda x.M$ — анонимная функция.
- Применение $M\,N$ — вызов функции M с аргументом N.
- Применение левоассоциативно. Абстракция правоассоциативна, её тело продолжается максимально вправо.

Примеры термов

x

$(x\ z)$

$(\lambda x.(x\ z))$

$((\lambda x.(x\ z))\ y)$

$(\lambda y.((\lambda x.(x\ z))\ y))$

$((\lambda y.((\lambda x.(x\ z))\ y))\ w)$

$(\lambda z.(\lambda w.((\lambda y.((\lambda x.(x\ z))\ y))\ w)))$

Термы (соглашения)

Общеприняты следующие соглашения:

- Внешние скобки опускаются.
- Применение левоассоциативно:

$F\,X\,Y\,Z$ обозначает $(((F\,X)\,Y)\,Z)$

- Абстракция правоассоциативна:

$\lambda x\,y\,z.M$ обозначает $\lambda x.(\lambda y.(\lambda z.M))$

- Тело абстракции простирается вправо насколько возможно:

$\lambda x.M\,N\,K$ обозначает $\lambda x.(M\,N\,K)$

Примеры термов

$$x = x$$

$$(x\ z) = x\ z$$

$$(\lambda x.(x\ z)) = \lambda x.x\ z$$

$$((\lambda x.(x\ z))\ y) = (\lambda x.x\ z)\ y$$

$$(\lambda y.((\lambda x.(x\ z))\ y)) = \lambda y.(\lambda x.x\ z)\ y$$

$$((\lambda y.((\lambda x.(x\ z))\ y))\ w) = (\lambda y.(\lambda x.x\ z)\ y)\ w$$

$$(\lambda z.(\lambda w.((\lambda y.((\lambda x.(x\ z))\ y))\ w))) = \lambda z\ w.(\lambda y.(\lambda x.x\ z)\ y)\ w$$

Свободные и связанные переменные

Определение

$\mathrm{FV}(\cdot)$ и $\mathrm{BV}(\cdot)$ задаются рекурсивно:

$$\mathrm{FV}(x) = \{x\} \qquad \mathrm{BV}(x) = \varnothing$$

$$\mathrm{FV}(M\,N) = \mathrm{FV}(M) \cup \mathrm{FV}(N) \qquad \mathrm{BV}(M\,N) = \mathrm{BV}(M) \cup \mathrm{BV}(N)$$

$$\mathrm{FV}(\lambda x.M) = \mathrm{FV}(M) \setminus \{x\} \qquad \mathrm{BV}(\lambda x.M) = \mathrm{BV}(M) \cup \{x\}$$

Пример

В $\lambda y.(\lambda x.x\,z)\,y\,w$ связаны x, y, а свободны z, w.

Упражнение: свободные и связанные переменные

Для каждого терма определите множества $\mathrm{FV}(\cdot)$ и $\mathrm{BV}(\cdot)$. Обратите внимание на случаи затенения (shadowing) переменных!

1. $\lambda x.x\,(\lambda y.x\;y\;z)\;u$
2. $\lambda f.\lambda x.f\;(g\;x)\;(\lambda y.h\;y)$
3. $\lambda x.(\lambda x.x\;y)\;x\;z$
4. $(\lambda a.\lambda b.a\;c\;(\lambda c.b\;c))\;d$
5. $\lambda p.\lambda q.p\;(\lambda p.q\;p\;r)\;s$

План лекции

1 Оргчасть и устройство курса

2 Мотивация

3 Введение в λ-исчисление

4 Формальный синтаксис и нотация

5 Редукции и отношения эквивалентности

6 Комбинаторы и кодирование данных

Бинарные отношения и отношение эквивалентности

Определение: бинарное отношение

Пусть A — множество. Бинарное отношение R на A — подмножество $A \times A$. Пишем $a\,R\,b$, если $(a, b) \in R$.

Определение: свойства отношений

Для $R \subseteq A \times A$:

- Рефлексивность: $\forall a \in A \quad a\,R\,a$
- Симметричность: $\forall a, b \in A \quad a\,R\,b \Rightarrow b\,R\,a$
- Транзитивность: $\forall a, b, c \in A \quad a\,R\,b \wedge b\,R\,c \Rightarrow a\,R\,c$

Определение: отношение эквивалентности

Эквивалентность — рефлексивное, симметричное и транзитивное отношение. Она разбивает множество на классы эквивалентности.

α-преобразование (шаг)

Определение: α-преобразование

$$\lambda x.M \to_\alpha \lambda y.M[x \mapsto y]$$

Здесь y — свежее имя: $y \notin \mathrm{Var}(M) \cup \{x\}$, где $\mathrm{Var}(M) = \mathrm{FV}(M) \cup \mathrm{BV}(M)$.

Замечание. $M[x \mapsto y]$ переименовывает свободные вхождения x в теле M. Внутренние связки λx и их области не затрагиваются.

α-преобразование: примеры

$\lambda x.\lambda y.x\,(\lambda x.y\;x) =_\alpha \lambda u.\lambda v.u\,(\lambda x.v\;x)$

$\lambda x.(\lambda x.x\;x) =_\alpha \lambda z.(\lambda u.u\;u)$

$\lambda y.x\;y \neq_\alpha \lambda x.x\;x$

Захват свободного x при переименовании $y \to x$ недопустим.

$\lambda x.\lambda y.x\;y =_\alpha \lambda y.\lambda x.y\;x$

Оба терма имеют ту же структуру связывания: $\lambda u.\lambda v.u\;v$.

$\lambda x.\lambda y.x\,(\lambda y.y) =_\alpha \lambda p.\lambda q.p\,(\lambda r.r)$

α-эквивалентность

Определение

Определим отношение $=_\alpha$ на Λ индукцией по структуре термов.

- Переменная: $x =_\alpha x$.
- Применение: если $M_1 =_\alpha N_1$ и $M_2 =_\alpha N_2$, то $M_1 M_2 =_\alpha N_1 N_2$.
- Абстракция: $\lambda x.M =_\alpha \lambda y.N$ тогда и только тогда, когда существует общее свежее имя z такое, что

$$M[x \mapsto z] =_\alpha N[y \mapsto z]$$

Свежесть: $z \notin \mathrm{Var}(M) \cup \mathrm{Var}(N) \cup \{x, y\}$.

Замечание 1. $[x \mapsto z]$ — переименование, определённое на предыдущих слайдах.

Замечание 2. Далее проверяется, что $=_\alpha$ — отношение эквивалентности.

Конвенция Барендрехта (формулировка)

Соглашение (BVC)

В рассуждениях о λ-термах работаем с α-эквивалентным представителем M' терма M таким, что:

1. Все связанные параметры используют попарно разные имена — нет затенения.
2. Ни одно связанное имя не совпадает ни с одним свободным:

$$\mathrm{BV}(M') \cap \mathrm{FV}(M') = \varnothing$$

Интуиция. Имена связанных переменных — временные ярлыки. Их можно переименовать так, чтобы они не мешали подстановкам и доказательствам.

Почему конвенция применима всегда

- α-эквивалентность позволяет переименовывать связанные переменные в свежие имена, не встречающиеся в рассматриваемых термах.
- Алфавит переменных бесконечен $\Rightarrow$ свежие имена всегда существуют.
- Для любого M существует $M' =_\alpha M$, удовлетворяющий BVC. Поэтому можно рассуждать «с точностью до α».

Следствие

Если y — свежее имя для $\lambda x.M$, то

$$\lambda x.M =_\alpha \lambda y.M[x \mapsto y]$$

Подстановка без захвата

Определение: подстановка $M[x := N]$

$M[x := N]$ — подстановка терма N вместо свободных вхождений x в M, задаваемая следующими правилами:

$$x[x := N] = N, \qquad y[x := N] = y, \quad \text{если } y \neq x$$

$$(P\,Q)[x := N] = (P[x := N])\,(Q[x := N])$$

$$(\lambda y.P)[x := N] = \begin{cases} \lambda y.P & y = x \\ \lambda y.(P[x := N]) & y \neq x,\ y \notin \mathrm{FV}(N) \\ \lambda z.((P[y := z])[x := N]) & y \neq x,\ y \in \mathrm{FV}(N) \end{cases}$$

В последней ветке: $z \notin \mathrm{FV}(P) \cup \mathrm{FV}(N) \cup \{x\}$.

β-редукция

Определение: β-редукция

$$(\lambda x.M)\,N \to_\beta M[x := N]$$

Определение: редекс

Редекс (reducible expression) — выражение вида $(\lambda x.M)\,N$.

- β-редукция — основной механизм вычисления в λ-исчислении.
- Моделирует применение функции к аргументу.
- Подстановка должна избегать захвата переменных.

Один шаг можно выполнить в любом подтерме, в том числе под λ.

Примеры β-редукции

Примеры

$(\lambda x.x)\,(\lambda y.y) \to_\beta \lambda y.y$

$(\lambda f.\lambda x.f\ x)\,(\lambda y.y) \to_\beta \lambda x.(\lambda y.y)\,x \to_\beta \lambda x.x$

$$
\begin{aligned}
&(\lambda f.\lambda g.\lambda x.f\ (g\ x))\,(\lambda y.y\ y)\,(\lambda z.z) \\
&\quad \to_\beta (\lambda g.\lambda x.(\lambda y.y\ y)\,(g\ x))\,(\lambda z.z) \\
&\quad \to_\beta \lambda x.(\lambda y.y\ y)\,((\lambda z.z)\,x) \\
&\quad \to_\beta \lambda x.(\lambda y.y\ y)\,x \\
&\quad \to_\beta \lambda x.x\ x
\end{aligned}
$$

Замкнутость относительно контекстов

Понятие: замкнутость относительно контекстов

Контекст — терм с одной «дыркой» $C[\cdot]$, строящийся по грамматике:

$$C ::= [\cdot] \mid C\,M \mid M\,C \mid \lambda x.C$$

Если $M \sim N$, то для любого C выполняется $C[M] \sim C[N]$.
Не путать с замкнутым термом: $\mathrm{FV}(M) = \varnothing$.

Пример

Пусть $C = \lambda z.[\cdot]\,z$. Тогда

$$C[\lambda x.x] =_\alpha C[\lambda y.y]$$

то есть $\lambda z.(\lambda x.x)\,z =_\alpha \lambda z.(\lambda y.y)\,z$.

β-эквивалентность

Определение: β-эквивалентность

β-эквивалентность $=_\beta$ — наименьшее отношение эквивалентности, замкнутое относительно контекстов и содержащее β-преобразование:

1. Основа: $(\lambda x.M)\,N \to_\beta M[x := N] \Rightarrow (\lambda x.M)\,N =_\beta M[x := N]$
2. Рефлексивность: $M =_\beta M$
3. Симметричность: $M =_\beta N \Rightarrow N =_\beta M$
4. Транзитивность: $M =_\beta N,\ N =_\beta L \Rightarrow M =_\beta L$
5. Совместимость: если $M =_\beta N$, то

$$\lambda x.M =_\beta \lambda x.N, \quad M\,L =_\beta N\,L, \quad L\,M =_\beta L\,N$$

Термы рассматриваются с точностью до α-эквивалентности.

β-эквивалентность: цепочки редукций

Определение: через конечную цепочку

Пишем $M \leftrightarrow_\beta N$, если $M \to_\beta N$ или $N \to_\beta M$ — один шаг в любую сторону. Тогда $M =_\beta N$ в точности при существовании конечной цепочки:

$$M_0 \leftrightarrow_\beta M_1 \leftrightarrow_\beta \cdots \leftrightarrow_\beta M_k, \quad M_0 = M, \quad M_k = N.$$

Замечание

Эквивалентно: $=_\beta$ — рефлексивно-симметрично-транзитивное замыкание $\to_\beta$ и наименьшая конгруэнция, содержащая $\to_\beta$.

β-эквивалентность: пример

Пусть $M \equiv (\lambda f.\lambda x.f\ (f\ x))\ g\ x$, $N \equiv g\ (g\ x)$. Тогда

$$M \to_\beta (\lambda x.g\ (g\ x))\ x \to_\beta g\ (g\ x) = N,$$

откуда $M =_\beta N$.

Ещё цепочка — шаги в обе стороны:

$$(\lambda x.x)\ t \to_\beta t \leftarrow_\beta (\lambda y.y)\ t.$$

Эти примеры иллюстрируют определение через конечные цепочки $\leftrightarrow_\beta$ и наличие общего редукта у показанных термов.

η-преобразование

Определение: η-преобразование

$$\lambda x.M\, x \to_\eta M, \quad \text{если } x \notin \mathrm{FV}(M).$$

Определение: η-эквивалентность

На α-классах отношение $=_\eta$ — наименьшая эквивалентность, замкнутая относительно контекстов и содержащая η-преобразование:

1. Основа: $\lambda x.M\, x =_\eta M$, если $x \notin \mathrm{FV}(M)$
2. Рефлексивность: $M =_\eta M$
3. Симметричность: $M =_\eta N \Rightarrow N =_\eta M$
4. Транзитивность: $M =_\eta N,\ N =_\eta L \Rightarrow M =_\eta L$
5. Совместимость: если $M =_\eta N$, то

$$\lambda x.M =_\eta \lambda x.N, \quad M\,L =_\eta N\,L, \quad L\,M =_\eta L\,N$$

Примеры η-эквивалентности

Экстенсиональность функций

η-эквивалентность выражает принцип экстенсиональности: абстракция $\lambda x.M\ x$ равна M, если x не входит свободно в M.

Примеры

$\lambda x.f\ x =_\eta f,$ если $x \notin \mathrm{FV}(f)$

$\lambda y.(\lambda z.z)\ y =_\eta \lambda z.z$

$\lambda a.(\lambda b.\lambda c.b)\ a =_\eta \lambda b.\lambda c.b$

$\lambda x.x\ x \neq_\eta x:$ условие нарушено, $x \in \mathrm{FV}(x)$.

Отношения эквивалентности

Определение: отношение эквивалентности

Эквивалентность — рефлексивное, симметричное и транзитивное отношение. Она разбивает множество на классы эквивалентности.

Контекст лекции

$=_\alpha$, $=_\beta$ и $=_\eta$ — отношения эквивалентности на термах. Мы работаем с точностью до этих эквивалентностей:

- По α: переименование связанных переменных несущественно.
- По β: сокращение редекса сохраняет класс терма.
- По η: можно удалить абстракцию $\lambda x.M\,x$ при $x \notin \mathrm{FV}(M)$.

Класс $[M]$ состоит из всех термов, эквивалентных M. Если β-нормальная форма существует, она единственна с точностью до α.

Отношения эквивалентности: пример

Примеры

$\lambda x.x =_\alpha \lambda y.y$

$(\lambda x.x)\, t =_\beta t$

$\lambda x.f\ x =_\eta f, \quad$ если $x \notin \mathrm{FV}(f)$

План лекции

1. Оргчасть и устройство курса
2. Мотивация
3. Введение в λ-исчисление
4. Формальный синтаксис и нотация
5. Редукции и отношения эквивалентности
6. **Комбинаторы и кодирование данных**

Комбинаторы

Определение

Комбинатор — замкнутый λ-терм M: $\mathrm{FV}(M) = \varnothing$. Множество всех замкнутых термов обозначается Λ_0.

$$I \coloneqq \lambda x.x$$

$$\omega \coloneqq \lambda x.x\ x$$

$$\Omega \coloneqq \omega\,\omega = (\lambda x.x\ x)\,(\lambda x.x\ x)$$

$$K \coloneqq \lambda x\ y.x$$

$$K\ I =_\beta \lambda x\ y.y$$

$$S \coloneqq \lambda f\ g\ x.f\ x\,(g\ x)$$

$$B \coloneqq \lambda f\ g\ x.f\,(g\ x)$$

Кодирование Чёрча: булеаны, пары, числа

Булеаны

$\text{true} \equiv \lambda t.\lambda f.t$

$\text{false} \equiv \lambda t.\lambda f.f$

$\text{if} \equiv \lambda b.\lambda x.\lambda y.b\ x\ y$

Пары

$\text{pair} \equiv \lambda x.\lambda y.\lambda p.p\ x\ y$

$\pi_1 \equiv \lambda p.p\ (\lambda x.\lambda y.x)$

$\pi_2 \equiv \lambda p.p\ (\lambda x.\lambda y.y)$

Натуральные (Чёрч)

$0 \equiv \lambda f.\lambda x.x$

$1 \equiv \lambda f.\lambda x.f\ x,\ \ldots$

$\text{succ} \equiv \lambda n.\lambda f.\lambda x.f\ (n\ f\ x)$

$\text{plus} \equiv \lambda m.\lambda n.\lambda f.\lambda x.m\ f\ (n\ f\ x)$

Что почитать

- Pierce, главы 5–7: нетипизированное λ-исчисление.
- Software Foundations: LF, Poly — функции и числа Чёрча. PLF, Stlc — подстановка и редукция.
- Более продвинутое: Barendregt — The Lambda Calculus.